

AcadémiePlus

Plateforme adaptative d'apprentissage
des mathématiques

Documentation technique complète

Architecture, fonctionnalités, base de données et moteur adaptatif

Projet	AcadémiePlus (<code>academieplus.lovable.app</code>)
Type	Application web SPA (React + Supabase)
Domaine	Soutien scolaire en mathématiques
Public	Élèves, Parents, Pédagogues, Administrateurs
Version doc.	1.0
Date	Juin 2026



Document destiné aux futures équipes de développement

Ce document décrit l'intégralité des choix techniques, des fonctionnalités
et de la structure de données du projet.

Table des matières

1	Introduction	4
1.1	Présentation du projet	4
1.2	Publics cibles et rôles	4
1.3	Niveaux scolaires couverts	4
1.4	Comment lire cette documentation	5
2	Architecture générale	6
2.1	Vue d'ensemble	6
2.2	Principes architecturaux	6
2.3	Organisation des dossiers	7
3	Pile technique et versions	8
3.1	Technologies principales	8
3.2	Bibliothèques notables	8
3.3	Outils de développement et qualité	8
3.4	Scripts npm	9
4	Authentification et gestion des rôles	10
4.1	Système d'authentification	10
4.2	Redirection intelligente après connexion	10
4.3	Stockage sécurisé des rôles	10
4.4	Création automatique de profil	11
4.5	Protection des routes	11
4.6	Liaison parent-enfant	11
5	Fonctionnalités (écran par écran)	12
5.1	Pages publiques	12
5.2	Onboarding	12
5.3	Espace élève	12
5.4	Espace parent	13
5.5	Espace administration	13
5.6	Espace éditorial (pédagogues / admins)	14
5.7	Composants pédagogiques clés	14
5.8	Assistant IA — chatbot	14
5.8.1	Quotas d'usage	14
5.9	Suivi parental et rapports	15
6	Moteur d'apprentissage adaptatif	16
6.1	Principe général	16
6.2	Niveau initial = test de positionnement	16
6.3	Étape 1 — Ajustement ELO par réponse	16
6.3.1	Pondérations	16
6.3.2	Score attendu et delta de base	16
6.3.3	Modulation par le temps de réponse	17
6.4	Étape 2 — Signaux comportementaux	17
6.4.1	Hésitation sans réponse soumise	17
6.5	Étape 3 — Score composite (calibration de l'IA)	17

6.6	Étape 4 — Niveau global pondéré	17
6.7	Étape 5 — Décroissance temporelle (oubli)	18
6.8	Enregistrement des réponses	18
6.9	Synthèse du calcul du niveau	18
7	Base de données Supabase	19
7.1	Utilisateurs et rôles	19
7.1.1	profiles	19
7.1.2	user_roles	19
7.1.3	parent_child_links	20
7.1.4	activity_logs	20
7.2	Contenu pédagogique	20
7.2.1	filieres	20
7.2.2	chapters	20
7.2.3	lessons	21
7.2.4	chapter_exercises	21
7.2.5	chapter_quizzes	21
7.2.6	exams	21
7.3	Suivi de l'apprentissage	22
7.3.1	student_scores	22
7.3.2	ai_generated_content	22
7.3.3	ai_lesson_comments	22
7.3.4	parent_reports	23
7.4	Assistant IA (chat)	23
7.4.1	chat_conversations	23
7.4.2	chat_usage	23
7.5	Abonnements et paiements	23
7.5.1	subscription_config	23
7.5.2	subscription_periods	24
7.5.3	student_subscriptions	24
7.5.4	activation_codes	24
7.5.5	payments	24
7.6	Fonctions de base de données	25
8	Modèle conceptuel de données (MCD)	26
8.1	Diagramme entité–association	26
8.2	Cardinalités principales	26
8.3	Regroupement fonctionnel des entités	27
9	Fonctions serveur (Edge Functions IA)	28
9.1	Fournisseurs et modèles d'IA	28
9.2	Catalogue des fonctions	28
9.3	Secrets configurés	29
10	Sécurité et règles d'accès (RLS)	30
10.1	Modèle de sécurité	30
10.2	Principes des politiques RLS	30
10.3	Bonnes pratiques appliquées	30

11 Déploiement et maintenance	32
11.1 Environnements	32
11.2 Variables d'environnement	32
11.3 Cycle de déploiement	32
11.4 Internationalisation	32
11.5 Tests	32
11.6 Conseils pour les prochaines équipes	33
Conclusion	33

Chapitre 1

Introduction

1.1 Présentation du projet

AcadémiePlus est une plateforme web de soutien scolaire spécialisée en **mathématiques**, destinée aux élèves du système éducatif algérien (du primaire jusqu'au baccalauréat). L'application combine du contenu de cours structuré, des exercices et quiz interactifs, un **moteur d'apprentissage adaptatif** qui ajuste la difficulté au niveau réel de l'élève, et un **assistant pédagogique IA** (chatbot) encadré par le programme scolaire.

🎯 Objectif pédagogique

Offrir à chaque élève un parcours *personnalisé* : le système mesure en permanence le niveau de l'élève (sur 100), détecte ses lacunes, génère des exercices calibrés et fournit des commentaires et recommandations automatiques. Les parents suivent la progression de leurs enfants via un tableau de bord dédié.

1.2 Publics cibles et rôles

La plateforme distingue quatre rôles applicatifs :

- 👤 **Élève (student)** — suit les cours, passe le test de positionnement, réalise exercices/quiz, dialogue avec l'IA.
- 👥 **Parent (parent)** — relie ses enfants à son compte et consulte leurs rapports de progression.
- 📖 **Pédagogue (pedago)** — rédige et gère le contenu des cours, exercices, quiz et examens (espace éditorial).
- 👤 **Administrateur (admin)** — supervise les utilisateurs, les abonnements, les codes d'activation et l'ensemble du contenu.

1.3 Niveaux scolaires couverts

Le contenu est organisé par niveau scolaire (énumération `school_level`) :

Code interne	Niveau
5eme_primaire	5 ^e année primaire
1ere_cem	1 ^{re} année CEM (collège)
2eme_cem	2 ^e année CEM
3eme_cem	3 ^e année CEM
4eme_cem	4 ^e année CEM (BEM)
premiere	1 ^{re} année secondaire
seconde	2 ^e année secondaire
terminale	3 ^e année secondaire (BAC)

Chaque niveau secondaire peut être subdivisé en **filières** (sciences, lettres, gestion, math-techniques...), ce qui détermine les chapitres affichés.

1.4 Comment lire cette documentation

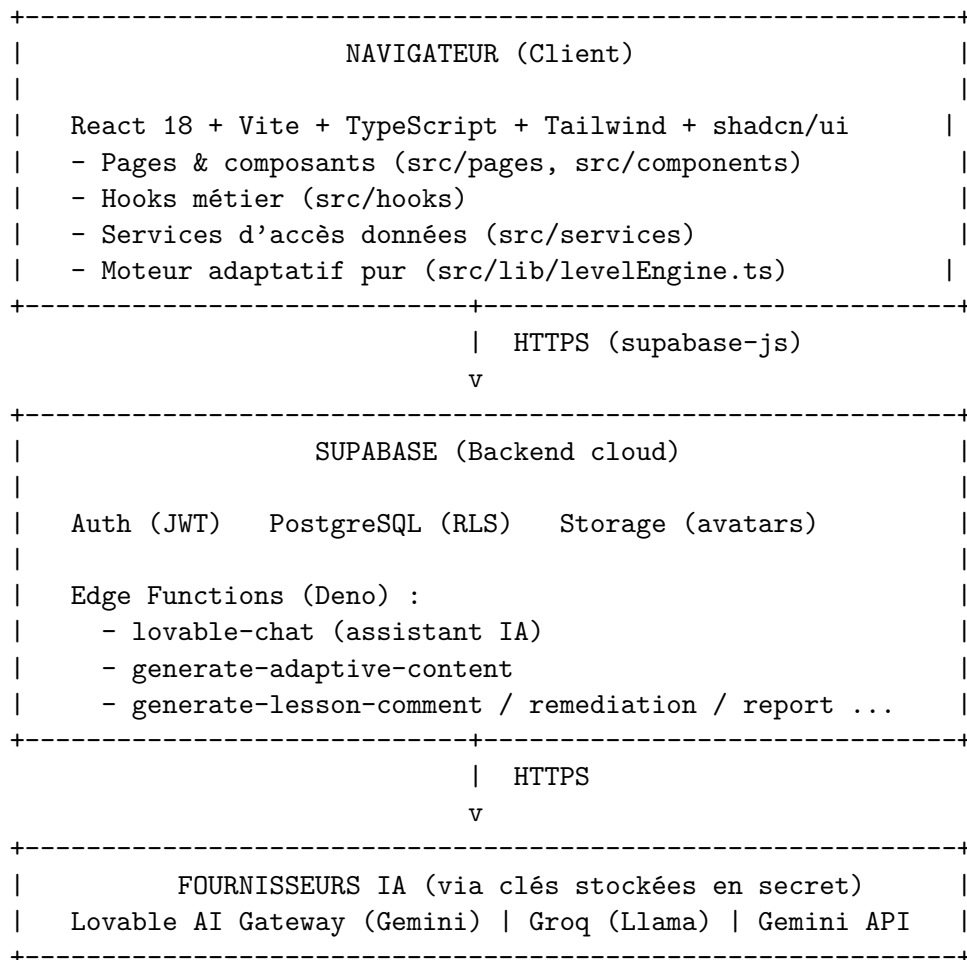
- Le **chapitre 2** décrit l'architecture globale.
- Le **chapitre 3** détaille la pile technique et les versions.
- Le **chapitre 4** explique l'authentification et la gestion des rôles.
- Le **chapitre 5** liste les fonctionnalités (pages) écran par écran.
- Le **chapitre 6** décrit le moteur adaptatif (calcul du niveau).
- Les **chapitres 7 et 8** documentent la base de données et le MCD.
- Le **chapitre 9** décrit les fonctions serveur (Edge Functions IA).
- Les **chapitres 10 et 11** traitent la sécurité et le déploiement.

Chapitre 2

Architecture générale

2.1 Vue d'ensemble

AcadémiePlus est une **application monopage (SPA)** React, hébergée et servie en statique, qui communique avec un **backend Supabase** (PostgreSQL + Auth + Storage + Edge Functions). Aucune partie serveur « maison » n'est maintenue : toute la logique métier vit soit dans le client React, soit dans des **fonctions Edge Deno** déployées sur l'infrastructure Supabase.



2.2 Principes architecturaux

- **Séparation des responsabilités** : les composants d'UI (`src/components`) ne contiennent pas la logique de calcul. Le calcul du niveau est isolé dans des *fonctions pures* (`src/lib/levelEngine.ts`), testables unitairement.
- **Accès données centralisé** : les appels Supabase passent par des *services* (`src/services`) et des *hooks* (`src/hooks`).
- **Sécurité côté base** : chaque table est protégée par des règles **RLS** (Row-Level Security) ; les rôles sont stockés dans une table dédiée `user_roles` (jamais sur le profil).

- **IA déportée** : les appels aux modèles d’IA se font exclusivement depuis des Edge Functions, afin de ne jamais exposer les clés au client.

2.3 Organisation des dossiers

Dossier	Contenu
src/pages/	Les pages routées (une par écran).
src/pages/editorial/	Espace de rédaction pour pédagogues/admins.
src/components/	Composants réutilisables (UI, course, dashboard...).
src/components/ui/	Bibliothèque shadcn/ui (boutons, dialogues...).
src/components/course/	Cur pédagogique : leçons, exercices, quiz.
src/hooks/	Hooks React métier (adaptatif, profil, abonnement...).
src/lib/	Logique pure : moteur de niveau, helpers, validation.
src/services/	Accès aux données Supabase encapsulés.
src/contexts/	Contextes React (authentification).
src/data/	Données statiques des programmes (chapitres par niveau).
src/i18n/	Internationalisation (français / arabe).
src/integrations/supabase/	Client Supabase et types générés.
supabase/functions/	Fonctions Edge (Deno) côté serveur.
supabase/migrations/	Migrations SQL versionnées de la base.

Chapitre 3

Pile technique et versions

3.1 Technologies principales

L'application repose sur une pile **React/Vite/TypeScript** moderne avec **Tailwind CSS** pour le style et **Supabase** pour le backend.

Technologie	Version	Rôle
React	18.3	Bibliothèque d'interface (composants, hooks).
Vite	5.4	Bundler / serveur de développement rapide.
TypeScript	5.8	Typage statique de tout le code.
Tailwind CSS	3.4	Système de design utilitaire (tokens sémantiques).
shadcn/ui (Radix)	—	Composants accessibles (dialogues, menus, onglets...).
React Router DOM	6.30	Routage côté client (SPA).
TanStack React Query	5.83	Cache et synchronisation des requêtes serveur.
Supabase JS	2.97	Client base de données / Auth / Storage / Fonctions.

3.2 Bibliothèques notables

Paquet	Usage dans le projet
i18next / react-i18next	Internationalisation FR/AR + détection de langue.
katex, react-katex, rehype-katex, remark-math	Rendu des formules mathématiques (L ^A T _E X) dans les leçons et le chat.
react-markdown, remark-gfm, rehype-raw	Rendu du contenu Markdown (cours, solutions, commentaires IA).
@tiptap/*	Éditeur de texte riche WYSIWYG pour l'espace éditorial.
recharts	Graphiques de progression (tableaux de bord, analytics).
framer-motion	Animations (compagnon-bot, transitions).
jspdf, html2canvas	Export PDF (cours, rapports parents).
react-hook-form + zod	Formulaires et validation de schémas.
@dnd-kit/*	Glisser-déposer (réordonnancement de sections éditoriales).
date-fns	Manipulation des dates.
lucide-react, react-icons	Jeux d'icônes.
sonner, vault, embla-carousel	Toasts, tiroirs, carrousels.

3.3 Outils de développement et qualité

— **ESLint 9** + **typescript-eslint** — qualité et cohérence du code.

- **Vitest 3** + `@testing-library/react` — tests unitaires/composants.
- **Playwright 1.57** — tests de bout en bout (navigateur).
- **PostCSS / Autoprefixer** — chaîne CSS.
- **@vitejs/plugin-react-swc** — compilation rapide via SWC.

3.4 Scripts npm

```
1 npm run dev          # serveur de developpement Vite
2 npm run build        # build de production
3 npm run build:dev    # build en mode developpement
4 npm run lint         # analyse ESLint
5 npm run test         # tests Vitest (une passe)
6 npm run test:watch   # tests en mode surveillance
```

Listing 3.1 – package.json — scripts

Note

Le projet est construit avec l'écosystème **Lovable**. Les fonctions Edge sont déployées automatiquement à chaque modification; le fichier `.env` est auto-rempli avec `VITE_SUPABASE_URL`, `VITE_SUPABASE_PUBLISHABLE_KEY` et `VITE_SUPABASE_PROJECT_ID`.

Chapitre 4

Authentification et gestion des rôles

4.1 Système d'authentification

L'authentification s'appuie sur **Supabase Auth** (jetons JWT). Le contexte React `src/contexts/AuthContext` centralise l'état de session :

- Au chargement, `supabase.auth.getSession()` récupère la session courante ; un abonnement `onAuthStateChange` maintient l'état à jour.
- Il expose `user`, `session`, `loading`, `signOut()`, `hasRole(role)` et `isAdmin()`.

4.2 Redirection intelligente après connexion

Après chaque connexion, le contexte oriente l'utilisateur selon son rôle et son état :

- **Sans rôle** → `/complete-profile` (compléter le profil).
- **Pédago / Admin** → `/liste-cours`.
- **Parent** → `/parent-dashboard`.
- **Élève sans test de positionnement** → `/learning-assessment` (test initial obligatoire).
- **Élève** (cas général) → `/cours/math`.

Détection du test de positionnement

La fonction `hasCompletedPlacementAssessment` vérifie l'existence d'une ligne `student_scores` « globale » (`lesson_id IS NULL`). Si elle est absente, l'élève est forcé de passer le test initial avant d'accéder aux cours. Une compatibilité ascendante évite de re-forcer les anciens comptes.

4.3 Stockage sécurisé des rôles

Conformément aux bonnes pratiques de sécurité, les rôles **ne sont jamais stockés sur le profil**. Ils résident dans une table dédiée `user_roles` et sont vérifiés côté base via une fonction SECURITY DEFINER :

```
1 CREATE OR REPLACE FUNCTION public.has_role(_user_id uuid, _role app_role)
2 RETURNS boolean LANGUAGE sql STABLE SECURITY DEFINER
3 SET search_path TO 'public' AS $$
4     SELECT EXISTS (
5         SELECT 1 FROM public.user_roles
6         WHERE user_id = _user_id AND role = _role
7     )
8 $$;
```

Listing 4.1 – Vérification de rôle sans récursion RLS

L'énumération des rôles est `app_role = {student, parent, admin, pedago}`.

4.4 Création automatique de profil

Un trigger `handle_new_user` s'exécute à l'inscription : il crée la ligne `profiles` (avec normalisation du niveau scolaire et des métadonnées) et insère le rôle dans `user_roles` à partir des métadonnées d'inscription.

4.5 Protection des routes

Le composant `src/components/ProtectedRoute.tsx` encapsule chaque route privée. Il accepte :

- `allowedRoles` — liste blanche de rôles autorisés ;
- `requireAdmin` — accès réservé aux administrateurs ;
- `blockAdmin` — interdit l'accès aux administrateurs (ex. page Compte).

4.6 Liaison parent–enfant

Un parent relie un enfant grâce à un **code de liaison** (`linking_code` dans `profiles`). La relation est stockée dans `parent_child_links` avec un statut (`pending`, `active`, `rejected`). La fonction `is_parent_of` sécurise l'accès des parents aux données de leurs enfants.

Chapitre 5

Fonctionnalités (écran par écran)

L'application est une SPA dont chaque écran correspond à une route déclarée dans `src/App.tsx`. Les routes sont protégées par rôle via `ProtectedRoute`. Cette section décrit chaque page par domaine.

5.1 Pages publiques

Page	Route	Description
Accueil	/	Page marketing (hero, atouts, tarifs, FAQ, CTA).
Authentification	/auth	Connexion / inscription (e-mail + mot de passe, choix du rôle, localisation, SSO).
Contact	/contact	Formulaire de contact.
Mentions légales	/mentions-legales	Page légale statique.
Confidentialité	/politique-confidentialite	Politique de confidentialité.
404	*	Page « non trouvée ».

5.2 Onboarding

Page	Route	Description
Compléter profil	/complete-profile	Complétion post-SSO (rôle, niveau, filière, localisation, date de naissance).
Test de positionnement	/learning-assessment	Test initial de l'élève : 5 questions adaptées au niveau scolaire ; le score initialise le niveau (ligne globale <code>student_scores</code>).
Onboarding Bot	/bot-onboarding	Questionnaire conversationnel du style d'apprentissage (visuel / textuel / pratique) animé par un bot bilingue.

5.3 Espace élève

Page	Route	Description
Tableau de bord	/dashboard	Synthèse de l'élève : niveau, activité récente, commentaires IA, recommandations.
Liste des cours	/liste-cours	Catalogue des matières / chapitres selon le niveau.
Cours	/cours/:subjectId	Interface centrale : arbre chapitres/leçons, contenu adaptatif, chatbot flottant, quiz et exercices, export PDF.

Page	Route	Description
Révision	/revision/:subjectId	Mode fiches (flashcards) de révision, cartes retournables.
Simulation	/simulation/:subjectId	Examen blanc chronométré (30 min), score et bilan par chapitre.
Examens (trimestre)	/exams	Sélection du trimestre.
Liste d'examens	/exams/list	Sujets d'examen par trimestre, mode chronométré.
Remédiation	/remediation	Activités ciblées sur les lacunes (3 exercices + 2 quiz via IA).
Compte	/account	Gestion du compte : avatar, abonnement (jours restants, pause), mot de passe.
Abonnements	/abonnements	Offres et état de l'abonnement, résiliation.
Paielement	/paiement	Tunnel de paiement.
Parrainage	/parrainage	Programme de parrainage (code, suivi, récompenses).
Profil	/profile	Vue profil (avatar, infos scolaires, statistiques).
Factures	/factures	Historique des factures.
Mes informations	/mes-informations	Édition des informations personnelles.
Mes données	/mes-donnees-personnelles	Chercher / suppression des données (RGPD).

5.4 Espace parent

Page	Route	Description
Tableau parent	/parent-dashboard	Liste des enfants liés, progression par matière, rapports IA, ajout d'enfant par code, export PDF.
Cours (vue parent)	/parent-cours/:childId	Vue en lecture seule de la progression d'un enfant.

5.5 Espace administration

Page	Route	Description
Admin	/admin	Gestion des utilisateurs (recherche, activation, suppression), statistiques, audit.
Abonnements admin	/admin/abonnements	Gestion des abonnements et des offres.
Analytics	/analytics	Journal d'activité, comptages, répartition des actions.
FAQ Admin	/faq-admin	Édition des FAQ de la page d'accueil.
Génération de contenu	/content-generation	Génération en masse d'exercices/quiz (progression en flux).
Éditeur de leçon	/lecon/:lessonId	Édition riche d'une leçon (TipTap, enrichissement IA, versions).

5.6 Espace éditorial (pédagogues / admins)

Page	Route	Description
Tableau éditorial	<code>/editorial</code>	Liste des chapitres/filières avec statut et date de modification.
Éditeur de cours	<code>/editorial/cours/:id</code>	Édition complète : sections, texte riche, formules, détection de conflit.
Prévisualisation	<code>.../preview</code>	Aperçu d'une leçon telle que vue par l'élève.
Historique	<code>.../historique</code>	Historique des versions, restauration.
Comparaison	<code>.../compare</code>	Comparaison de deux versions (diff).
Révision	<code>/editorial/revision</code>	File de relecture / validation.
Médiathèque	<code>/editorial/mediatheque</code>	Bibliothèque de médias (Storage).
Gestion équipe	<code>/editorial/equipe</code>	Gestion des membres de l'équipe pédagogique.

5.7 Composants pédagogiques clés

- `AdaptiveLesson` — affiche la leçon avec bascule *vidéo* / *texte* selon le style d'apprentissage ; rendu KaTeX des formules.
- `LessonActivityTabs` — organise les activités en trois étapes pédagogiques : *Découvrir*, *Comprendre* et *Approfondir* (contenu IA). L'accès à l'approfondissement est conditionné à l'abonnement ou à 3 bonnes réponses.
- `ChapterMathExercises` — exercices à réponse libre, clavier mathématique, indices, validation serveur, verrou 3s sur erreur, détection d'hésitation.
- `ChapterMathQuiz` — quiz à choix multiples avec confirmation, suivi du temps et bilan final.
- `ChatBot` — assistant IA flottant et déplaçable (texte, image, saisie vocale FR/AR), encadré par le programme.

5.8 Assistant IA — chatbot

Le chatbot (`src/components/ChatBot.tsx`) appelle la fonction `Edge lovable-chat`. Il est :

- **contextualisé** : reçoit le contenu du chapitre courant et la liste des leçons (pour des liens de navigation) ;
- **cadré** : répond uniquement en mathématiques, dans le programme ;
- **multimodal** : accepte texte, images et saisie vocale ;
- **évaluateur** : peut poser des exercices ; un marqueur caché `[[EVAL:correct|difficulté|concept]]` permet d'alimenter directement le moteur de niveau (`src/lib/chatEvalTracker.ts`).

5.8.1 Quotas d'usage

Le hook `src/hooks/useChatLimits.ts` applique des quotas pour les comptes gratuits : **10 messages** et **3 images** par jour (table `chat_usage`). Les abonnés actifs ont un accès illimité. L'historique est sauvegardé (avec anti-rebond de 2s) dans `chat_conversations`, en retirant les données d'image pour alléger le stockage.

5.9 Suivi parental et rapports

Les parents lient leurs enfants via un code, puis consultent des **rapports de progression** générés par IA (fonction `generate-parent-report`). Une tâche planifiée (`scheduled-parent-reports`) produit ces rapports périodiquement et envoie des alertes en cas d'inactivité prolongée de l'enfant.

Chapitre 6

Moteur d'apprentissage adaptatif

Ce chapitre répond directement à la question « *comment est calculé le niveau de l'élève ?* ». Toute la logique de calcul est concentrée dans des fonctions pures de `src/lib/levelEngine.ts`, ce qui la rend testable et indépendante de l'interface.

6.1 Principe général

Chaque élève possède un **niveau** compris entre **5** et **100** pour chaque leçon (et un niveau global par chapitre). Le niveau évolue après chaque réponse selon un modèle inspiré du système **ELO**, pondéré par la difficulté, le temps de réponse et des **signaux comportementaux**.

6.2 Niveau initial = test de positionnement

À la création de son compte, l'élève passe un **test diagnostic** (`src/pages/LearningAssessment.tsx`). Son score initial est enregistré dans une ligne globale de `student_scores` (`lesson_id = NULL`).

i Pas de 50 par défaut

Le niveau initial **n'est plus une valeur arbitraire de 50**. La fonction `getPlacementLevel(userId)` (`src/lib/initialLevel.ts`) lit le `current_level` de la ligne de positionnement et l'utilise pour amorcer chaque nouvelle leçon. À défaut (ancien compte), la valeur 50 sert de repli.

6.3 Étape 1 — Ajustement ELO par réponse

La fonction `computeDelta` calcule la variation Δ du niveau après chaque réponse.

6.3.1 Pondérations

Difficulté	Poids	Temps médian attendu (s)
1	0,6	30
2	0,8	45
3	1,0	60
4	1,3	90
5	1,6	120

6.3.2 Score attendu et delta de base

Le « score attendu » suit une logistique de type ELO :

$$E = \frac{1}{1 + 10^{(20 \cdot d - L)/40}}$$

où d est la difficulté et L le niveau courant. Puis :

$$\Delta = \begin{cases} \text{clamp}(\text{round}(3(1 - E) \cdot w), 1, 5) & \text{si correct} \\ \text{clamp}(\text{round}(-2E \cdot w), -5, -1) & \text{si faux} \end{cases}$$

avec w le poids de difficulté.

6.3.3 Modulation par le temps de réponse

- Réponse correcte *très rapide* ($< 0,6 \times$ médian) : $\Delta \times 1,2$.
- Réponse correcte *très lente* ($> 2 \times$ médian) : $\Delta \times 0,8$.
- Réponse fausse après un temps très long (> 120 s) : $\Delta \times 1,3$.

6.4 Étape 2 — Signaux comportementaux

Le moteur enrichit le delta en tenant compte de la manière dont l'élève a répondu :

Signal	Effet sur le gain/la perte
Tentatives multiples (réussite après erreurs)	gain $\times 0,85^{(\text{tentatives}-1)}$ (plancher +1).
Indice préventif (consulté avant toute réponse)	gain $\times 0,70$ (−30%).
Indice curatif (consulté après une erreur)	gain $\times 0,40$ (−60%).
Abandon avec indice curatif	perte $\times 1,3$.

Le delta final est re-borné à $[+1, +6]$ (réussite) ou $[-7, -1]$ (échec).

6.4.1 Hésitation sans réponse soumise

La fonction `hesitationAdjustment(kind, difficulty)` gère deux cas :

- « **timeout** » (exercice ouvert > 1 min sans réponse puis quitté) : -1 , ou -2 si difficulté ≥ 4 .
- « **erase** » (saisie puis effacement total puis sortie) : 0 sur le niveau (enregistré uniquement dans le profil pour affiner le suivi).

Ces drapeaux sont cumulés dans le champ JSON `assessment_data` (`hesitation_timeout`, `hesitation_erase`).

6.5 Étape 3 — Score composite (calibration de l'IA)

Pour calibrer la difficulté du contenu généré, on calcule un **score composite** (`computeComposite`) :

$$C = 0,40 \times \text{taux pondéré} + 0,35 \times L + 0,25 \times \text{taux quiz}$$

borné à $[5, 100]$. Ce score sert de `difficulty_level` transmis aux fonctions de génération de contenu adaptatif.

6.6 Étape 4 — Niveau global pondéré

`computeGlobal` agrège les niveaux de leçon en un niveau de chapitre, pondéré par le nombre de réponses :

$$L_{\text{global}} = \frac{\sum_i L_i \times n_i}{\sum_i n_i}$$

Elle identifie aussi la leçon la plus faible / la plus forte et signale une *lacune critique* dès qu'une leçon descend sous 30.

6.7 Étape 5 — Décroissance temporelle (oubli)

`applyDecay` simule l'oubli en cas d'inactivité :

- ≤ 7 jours : aucune décroissance.
- > 7 jours : facteur = $\max(0,10, 1 - 0,01 \times (\text{jours} - 7))$.

La décroissance est appliquée à l'ouverture d'une leçon et persistée en base.

6.8 Enregistrement des réponses

- `src/hooks/useAdaptiveContent.ts` — gère le contenu adaptatif, les compteurs de session, et appelle `recordAnswer` (qui applique `computeDelta` puis met à jour `student_scores`).
- `src/lib/recordActivityAnswer.ts` — fonction partagée permettant aux activités *de chapitre* (non adaptatives) d'alimenter aussi le moteur de niveau, avec les mêmes signaux comportementaux.
- `src/lib/chatEvalTracker.ts` — suit les évaluations issues du chat.

💡 Comportement face à une mauvaise réponse

Dans les exercices, une réponse fausse **n'affiche jamais la bonne réponse automatique** : l'interface se verrouille en rouge pendant 3 secondes, puis l'élève peut réessayer. La solution détaillée n'apparaît que s'il clique volontairement sur le bouton « Voir la solution détaillée » (ce qui compte alors comme un abandon). Dans les quiz, l'élève sélectionne une option puis confirme via le bouton « Confirmer ».

6.9 Synthèse du calcul du niveau

1. **Niveau initial** = score du test de positionnement.
2. **Après chaque réponse** : Δ ELO (difficulté + temps) modulé par les signaux comportementaux $\rightarrow$ nouveau `current_level`.
3. **Niveau de chapitre** = moyenne pondérée des niveaux de leçon.
4. **Score composite** = calibration de la difficulté du contenu IA.
5. **Décroissance** = -1% /jour au-delà de 7 jours d'inactivité.

Chapitre 7

Base de données Supabase

La base est une instance **PostgreSQL** gérée par Supabase. Elle comporte **21 tables** dans le schéma **public**, protégées par RLS. Cette section décrit chaque table, ses colonnes principales et son rôle. Les énumérations utilisées sont :

- `app_role` : `student`, `parent`, `admin`, `pedago`;
- `school_level` : `5eme_primaire`, `1ere_cem`, `2eme_cem`, `3eme_cem`, `4eme_cem`, `premiere`, `seconde`, `terminale`;
- `link_status` : `pending`, `active`, `rejected`.

Note

Les relations entre tables sont *logiques* (par `user_id`, `chapter_id`, `lesson_id`...) et non matérialisées par des clés étrangères strictes vers `auth.users` (recommandation Supabase). Le MCD du chapitre 8 les illustre.

7.1 Utilisateurs et rôles

7.1.1 profiles

Profil applicatif de chaque utilisateur (extension de `auth.users`).

Colonne	Type	Description
<code>id</code>	<code>uuid</code>	Identifiant (= <code>id auth</code>).
<code>email</code>	<code>text</code>	Adresse e-mail.
<code>email_verified</code>	<code>boolean</code>	E-mail confirmé.
<code>first_name</code> / <code>last_name</code>	<code>text</code>	Nom et prénom.
<code>phone</code>	<code>text</code>	Téléphone.
<code>avatar_url</code>	<code>text</code>	Photo de profil (bucket <code>avatars</code>).
<code>school_level</code>	<code>enum</code>	Niveau scolaire.
<code>filiere</code>	<code>text</code>	Filière (secondaire).
<code>linking_code</code>	<code>text</code>	Code de liaison parent-enfant.
<code>is_active</code>	<code>boolean</code>	Compte actif.
<code>date_of_birth</code>	<code>date</code>	Date de naissance.
<code>wilaya</code> / <code>ville</code> / <code>ecole</code>	<code>text</code>	Localisation et établissement.

7.1.2 user_roles

Rôles applicatifs (séparés du profil pour la sécurité).

Colonne	Type	Description
<code>id</code>	<code>uuid</code>	Identifiant.
<code>user_id</code>	<code>uuid</code>	Utilisateur concerné.

Colonne	Type	Description
role	app_role	Rôle (student/parent/admin/pedago).
created_at	timestampz	Date d'attribution.

7.1.3 parent_child_links

Liaison entre un parent et un enfant.

Colonne	Type	Description
parent_id	uuid	Parent.
child_id	uuid	Enfant.
status	link_status	État (pending/active/rejected).

7.1.4 activity_logs

Journal d'audit des actions (via fonction `log_activity`).

Colonne	Type	Description
user_id	uuid	Auteur de l'action.
action	text	Code de l'action.
details	jsonb	Détails contextuels.
ip_address	text	Adresse IP (optionnelle).

7.2 Contenu pédagogique

7.2.1 filieres

Filières disponibles par niveau scolaire.

Colonne	Type	Description
school_level	enum	Niveau associé.
code	text	Code court (ex. <code>sciences</code>).
name / name_ar	text	Libellé FR / AR.

7.2.2 chapters

Chapitres de cours, organisés par niveau, filière et matière.

Colonne	Type	Description
school_level	enum	Niveau scolaire.
filiere_id	uuid	Filière (nullable).
title / title_ar	text	Titre FR / AR.
description	text	Description.
subject	text	Matière (<code>math</code>).
order_index	integer	Ordre d'affichage.

7.2.3 lessons

Leçons appartenant à un chapitre.

Colonne	Type	Description
chapter_id	uuid	Chapitre parent.
title / title_ar	text	Titre FR / AR.
content	text	Contenu (Markdown/HTML).
video_url	text	Vidéo associée.
order_index	integer	Ordre.

7.2.4 chapter_exercises

Exercices d'entraînement (réponse libre).

Colonne	Type	Description
chapter_id / lesson_id	uuid	Rattachement.
title	text	Titre de l'exercice.
statement	text	Énoncé.
expected_answer	text	Réponse attendue.
accepted_answers	jsonb	Variantes acceptées.
solution	text	Solution détaillée.
hint	text	Indice.
difficulty	integer	Difficulté (1–5).

7.2.5 chapter_quizzes

Questions de quiz (choix multiples).

Colonne	Type	Description
chapter_id / lesson_id	uuid	Rattachement.
question	text	Énoncé de la question.
options	jsonb	Liste des choix.
correct_answer	text	Bonne réponse.
explanation	text	Explication.
hint	text	Indice.
difficulty	integer	Difficulté (1–5).

7.2.6 exams

Examens / sujets par trimestre.

Colonne	Type	Description
school_level	enum	Niveau.
trimester	integer	Trimestre.
title / title_ar	text	Titre.
content	jsonb	Contenu structuré.
duration_minutes	integer	Durée.

Colonne	Type	Description
subject	text	Matière.

7.3 Suivi de l'apprentissage

7.3.1 student_scores

Cur du moteur adaptatif : niveau et statistiques par élève/leçon/chapitre.

Colonne	Type	Description
user_id	uuid	Élève.
chapter_id / lesson_id	uuid	Portée (leçon, chapitre, ou globale si NULL).
current_level	integer	Niveau courant (5–100).
reading/quiz/exercise_time	integer/seconds	Temps passé par type.
total_answers	integer	Nombre total de réponses.
correct_answers	integer	Réponses correctes.
accuracy_rate	numeric	Taux de réussite (%).
streak	integer	Série de bonnes réponses.
assessment_data	jsonb	Drapeaux d'hésitation, données du test.
periodic_advice	jsonb	Conseils périodiques.
advice_seen	boolean	Conseil consulté.
report_first_shown_at	timestamptz	Premier affichage du rapport.

7.3.2 ai_generated_content

Contenu (quiz/exercices/révision) généré par IA et calibré par niveau.

Colonne	Type	Description
user_id	uuid	Élève destinataire.
chapter_id / lesson_id	uuid	Rattachement.
content_type	text	quiz / exercise / revision.
content	jsonb	Items générés.
difficulty_level	integer	Niveau de calibration (composite).

7.3.3 ai_lesson_comments

Commentaires IA personnalisés après une session d'étude.

Colonne	Type	Description
user_id	uuid	Élève.
lesson_id / chapter_id	uuid	Leçon/chapitre concernés.
level_before / level_after / level_delta	integer	Évolution du niveau.
weak_concepts / strong_concepts	jsonb	Lacunes et acquis détectés.

Colonne	Type	Description
message	text	Commentaire (arabe, Markdown).
link_url	text	Lien vers la leçon.

7.3.4 parent_reports

Rapports de progression destinés aux parents.

Colonne	Type	Description
parent_id / child_id	uuid	Parent et enfant.
period_start / period_end	timestampz	Période couverte.
report_data	jsonb	Données brutes.
global_success_rate	numeric	Taux de réussite global.
global_level	integer	Niveau global.
strong_chapters / weak_chapters	jsonb	Forces et faiblesses.
ai_recommendations	text	Recommandations IA.
/ summary report_type	text	Type de rapport.

7.4 Assistant IA (chat)

7.4.1 chat_conversations

Historique des conversations avec l'assistant IA.

Colonne	Type	Description
user_id	uuid	Propriétaire.
chapter_id	uuid	Contexte (nullable).
title	text	Titre de la conversation.
messages	jsonb	Fil de messages.

7.4.2 chat_usage

Quotas d'utilisation quotidienne du chat (messages/images).

Colonne	Type	Description
user_id	uuid	Utilisateur.
usage_date	date	Jour.
message_count	integer	Nombre de messages.
image_count	integer	Nombre d'images.

7.5 Abonnements et paiements

7.5.1 subscription_config

Configuration des offres (prix individuel/famille).

Colonne	Type	Description
plan_type	text	Type d'offre.
price_single / price_family	integer	Tarifs.
label	text	Libellé.
is_active	boolean	Offre active.

7.5.2 subscription_periods

Périodes de validité des abonnements.

Colonne	Type	Description
label	text	Libellé.
start_date / end_date	date	Bornes.
is_active	boolean	Période active.

7.5.3 student_subscriptions

Abonnement actif d'un élève (décompte de jours).

Colonne	Type	Description
user_id	uuid	Élève.
activation_code_id	uuid	Code utilisé.
plan_type	text	Type d'offre.
total_days / days_used	integer/numeric	Crédit et consommation.
is_paused / paused_at	boolean/ts	Mise en pause.
started_at / last_tick_at	timestampz	Démarrage et dernier décompte.

7.5.4 activation_codes

Codes prépayés d'activation d'abonnement.

Colonne	Type	Description
code	text	Code unique.
plan_type	text	Offre liée.
is_family	boolean	Offre famille.
status	text	État (free/used...).
created_by / used_by	uuid	Créateur / utilisateur.
payment_id	uuid	Paieement associé.

7.5.5 payments

Historique des paiements.

Colonne	Type	Description
user_id	uuid	Payeur.
plan_type / plan_label	text	Offre.
amount	integer	Montant.
is_family / children_count	bool/int	Offre famille.
period_id	uuid	Période.
status	text	État du paiement.

7.6 Fonctions de base de données

Fonction	Rôle
has_role(uid, role)	Vérifie un rôle (SECURITY DEFINER, anti-récursion RLS).
is_parent_of(parent, child)	Vérifie une liaison parent-enfant active.
user_has_any_role(uid)	Indique si l'utilisateur a au moins un rôle.
handle_new_user()	Trigger : crée profil + rôle à l'inscription.
handle_updated_at()	Trigger : met à jour <code>updated_at</code> .
log_activity(uid, action, details)	Insère une ligne d'audit.
generate_activation_code()	Génère un code d'activation aléatoire.
check_exercise_answer(id, rep)	Valide une réponse d'exercice (comparaison normalisée exacte).
check_quiz_answer(id, rep)	Valide une réponse de quiz.
get_student_exercises / get_student_quizzes	Renvoie les exercices/quiz d'un chapitre/leçon.

✓ Validation exacte des réponses

La fonction `check_exercise_answer` normalise la saisie (minuscules, suppression des espaces) puis effectue une **comparaison exacte** avec la réponse attendue et les variantes acceptées. Cela évite qu'une saisie partielle (ex. « 1 ») soit acceptée pour une réponse « 12 ».

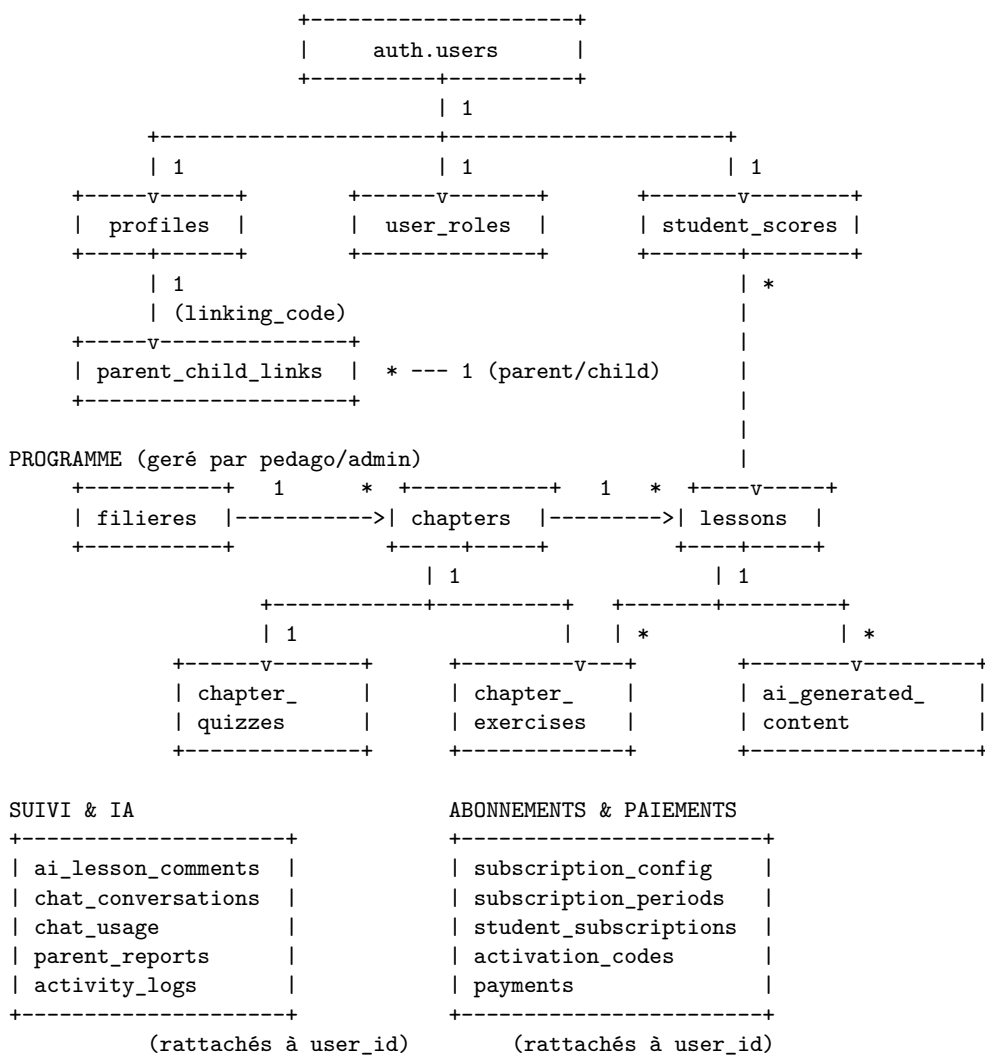
Chapitre 8

Modèle conceptuel de données (MCD)

Ce chapitre présente le modèle conceptuel des données. Les associations sont *logiques* : elles reposent sur des colonnes d'identifiant (`user_id`, `chapter_id`, `lesson_id`...) plutôt que sur des contraintes de clés étrangères strictes vers `auth.users` (bonne pratique Supabase).

8.1 Diagramme entité–association

Le schéma ci-dessous résume les principales entités et leurs cardinalités.



8.2 Cardinalités principales

Entité A	Card.	Entité B	Clé logique
profiles	1 — 1	auth.users	id
user_roles	* — 1	auth.users	user_id

Entité A	Card.	Entité B	Clé logique
parent_child_links	* — 1	profiles (parent)	parent_id
parent_child_links	* — 1	profiles (enfant)	child_id
filieres	1 — *	chapters	filiere_id
chapters	1 — *	lessons	chapter_id
chapters	1 — *	chapter_exercises	chapter_id
chapters	1 — *	chapter_quizzes	chapter_id
lessons	1 — *	chapter_exercises	lesson_id
lessons	1 — *	chapter_quizzes	lesson_id
profiles (élève)	1 — *	student_scores	user_id
lessons	1 — *	student_scores	lesson_id
profiles (élève)	1 — *	ai_generated_content	user_id
profiles (élève)	1 — *	ai_lesson_comments	user_id
profiles	1 — *	chat_conversations	user_id
parent + enfant	1 — *	parent_reports	parent_id, child_id
profiles (élève)	1 — *	student_subscriptions	user_id
activation_codes	1 — 1	student_subscriptions	activation_code_id
profiles	1 — *	payments	user_id

8.3 Regroupement fonctionnel des entités

- **Identité & accès** : profiles, user_roles, parent_child_links, activity_logs.
- **Programme** : filieres, chapters, lessons, chapter_exercises, chapter_quizzes, exams.
- **Apprentissage adaptatif** : student_scores, ai_generated_content, ai_lesson_comments.
- **Assistant IA** : chat_conversations, chat_usage.
- **Suivi parental** : parent_reports.
- **Commerce** : subscription_config, subscription_periods, student_subscriptions, activation_codes, payments.

Fonctions serveur (Edge Functions IA)

Toute la logique faisant appel à l'intelligence artificielle est déportée dans des **Edge Functions Deno** hébergées par Supabase (`supabase/functions/`). Ce choix garantit que les clés d'API ne sont **jamais** exposées au navigateur : elles sont lues côté serveur via `Deno.env.get(...)`.

9.1 Fournisseurs et modèles d'IA

Fournisseur	Usage
Lovable AI Gateway	Passerelle principale vers Google Gemini (<code>google/gemini-2.5-flash</code> , <code>google/gemini-3-flash-preview</code>). Clé <code>LOVABLE_API_KEY</code> .
Groq	Modèle de repli rapide <code>llama-3.3-70b-versatile</code> . Clé <code>GROQ_API_KEY</code> .
Google Gemini (direct)	Génération en masse via <code>GEMINI_API_KEY</code> / <code>GEMINI_API_KEY_2</code> .
OpenRouter / Cloudflare AI	Clés additionnelles disponibles (<code>OPENROUTER_API_KEY</code> , <code>CLOUDFLARE_AI_API_KEY</code>).



Stratégie de repli

Plusieurs fonctions tentent d'abord Gemini (via la passerelle) puis basculent automatiquement sur Groq (Llama) en cas d'erreur, afin de garantir la disponibilité du service.

9.2 Catalogue des fonctions

Fonction	Rôle
<code>lovable-chat</code>	Assistant pédagogique IA (chatbot). Réponses contextualisées et strictement encadrées par le programme de mathématiques. Gemini avec repli Llama.
<code>generate-adaptive-content</code>	Génère quiz / exercices / révisions calibrés au niveau composite de l'élève (Gemini 3 flash).
<code>generate-lesson-comment</code>	Produit un commentaire IA personnalisé en arabe après une session d'étude (forces/lacunes, conseils).
<code>generate-remediation</code>	Génère des activités de remédiation ciblées <i>uniquement</i> sur les lacunes détectées de l'élève.
<code>generate-chapter-quizzes</code>	Génère des quiz pour tout un chapitre (Gemini + repli Llama).
<code>generate-chapter-revision</code>	Génère des fiches de révision de chapitre.
<code>generate-parent-report</code>	Construit le rapport de progression destiné aux parents (synthèse + recommandations IA).

Fonction	Rôle
<code>scheduled-parent-reports</code>	Tâche planifiée (<code>pg_cron</code>) générant quotidiennement les rapports pour chaque liaison parent-enfant active.
<code>enrich-chapter-lessons</code>	Enrichit le contenu des leçons d'un chapitre.
<code>bulk-generate-lesson-content</code>	Génère en masse 10 exercices + 10 quiz pour une leçon et les insère en base.
<code>bulk-generate-all-terminale</code>	Génération par lot pour toutes les leçons de Terminale Sciences.
<code>bulk-gen-terminale-gemini</code>	Variante de génération en masse via l'API Gemini directe (<code>GEMINI_API_KEY_2</code>).
<code>delete-user-account</code>	Suppression complète et sécurisée d'un compte utilisateur (privilèges <code>service_role</code>).

9.3 Secrets configurés

Les secrets suivants sont disponibles côté Edge Functions (jamais exposés au client) :

- IA : `LOVABLE_API_KEY`, `GROQ_API_KEY`, `GEMINI_API_KEY`, `GEMINI_API_KEY_2`, `OPENROUTER_API_KEY`, `CLOUDFLARE_AI_API_KEY`, `CLOUDFLARE_ACCOUNT_ID`.
- Supabase : `SUPABASE_URL`, `SUPABASE_DB_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_ANON_KEY`, `SUPABASE_PUBLISHABLE_KEY`, `SUPABASE_JWKS`...

Chapitre 10

Sécurité et règles d'accès (RLS)

10.1 Modèle de sécurité

La sécurité repose sur trois piliers :

1. **Authentification JWT** via Supabase Auth.
2. **Rôles séparés** dans `user_roles`, vérifiés par des fonctions `SECURITY DEFINER` (`has_role`, `is_parent_of`) — évite l'élévation de privilèges et la récursion RLS.
3. **Row-Level Security (RLS)** activée sur *toutes* les tables du schéma `public`, complétée par des `GRANT` explicites.

10.2 Principes des politiques RLS

Domaine	Règle d'accès
Données personnelles	Chaque utilisateur ne voit/modifie que ses propres lignes (<code>auth.uid() = user_id</code>) : <code>student_scores</code> , <code>ai_generated_content</code> , <code>ai_lesson_comments</code> , <code>chat_conversations</code> , <code>chat_usage</code> .
Contenu pédagogique	Lecture réservée aux utilisateurs authentifiés ; gestion (écriture) réservée aux <code>pedago</code> et <code>admin</code> : <code>chapters</code> , <code>lessons</code> , <code>chapter_exercises</code> , <code>chapter_quizzes</code> , <code>exams</code> , <code>filieres</code> .
Suivi parental	Un parent accède aux données de ses enfants <i>uniquement</i> si une liaison active existe (<code>is_parent_of</code>) : <code>parent_reports</code> , <code>ai_lesson_comments</code> (enfants).
Administration	Les administrateurs disposent de politiques « manage » (ALL) sur les codes, abonnements, journaux, etc.
Audit	<code>activity_logs</code> : insertion par utilisateurs authentifiés, lecture par le propriétaire et les administrateurs.

10.3 Bonnes pratiques appliquées

- **Aucun secret en base** : les clés d'API vivent dans les secrets Supabase, jamais dans les tables ni dans le client.
- **service_role jamais exposé** au navigateur ; utilisé seulement à l'intérieur des Edge Functions.
- **Validation des réponses côté serveur** via des fonctions `SECURITY DEFINER` (`check_exercise_answer`, `check_quiz_answer`) — l'élève ne peut pas connaître la réponse en inspectant le client.
- **GRANT explicites** pour chaque table conformément aux exigences de PostgREST.

⚠ Point de vigilance

Les rôles ne doivent jamais migrer vers la table `profiles`. Toute vérification d'autorisation côté client (`hasRole`) est un confort d'UX ; la véritable barrière de sécurité reste la **RLS** et

les fonctions `SECURITY DEFINER` côté base.

Chapitre 11

Déploiement et maintenance

11.1 Environnements

Environnement	URL
Aperçu (preview)	https://id-preview--d3d48394-d747-4de9-844c-541a6a3403d6.lovable.app
Production	https://academieplus.lovable.app

Le projet Supabase associé porte la référence `lfothlxoixayjiytwwqa`.

11.2 Variables d'environnement

Le fichier `.env` est rempli automatiquement avec :

```
1 VITE_SUPABASE_URL=...
2 VITE_SUPABASE_PUBLISHABLE_KEY=... # cle publique (anon) - OK cote client
3 VITE_SUPABASE_PROJECT_ID=...
```

La clé publique (`anon`) est volontairement publique ; la protection réelle des données provient de la RLS. Les clés *privées* restent dans les secrets Supabase.

11.3 Cycle de déploiement

- **Frontend** : build Vite (`npm run build`) servi en statique.
- **Edge Functions** : déploiement **automatique** à chaque modification (aucune commande manuelle).
- **Migrations** : les fichiers SQL de `supabase/migrations/` sont versionnés et appliqués via l'outil de migration.

11.4 Internationalisation

L'application est bilingue **français / arabe** via `i18next` (`src/i18n/`). Les libellés sont stockés dans `src/i18n/locales/fr.json` et `src/i18n/locales/ar.json`. L'interface élève est majoritairement en **arabe** (RTL), tandis que les espaces d'administration et éditorial sont en français.

11.5 Tests

- **Unitaires / composants** : Vitest + Testing Library (`src/test/`). Le moteur de niveau (`src/lib/levelEngine.ts`) étant constitué de fonctions pures, il est idéal à couvrir par des tests.
- **Bout en bout** : Playwright (`playwright.config.ts`).

11.6 Conseils pour les prochaines équipes

1. **Logique de niveau** : toute évolution du calcul du niveau doit passer par `src/lib/levelEngine.ts` (fonctions pures, testables) plutôt que d'être dispersée dans les composants.
2. **Accès données** : privilégier les services (`src/services`) et hooks (`src/hooks`) existants plutôt que des appels Supabase directs dans les composants.
3. **Sécurité** : ne jamais déplacer les rôles dans `profiles` ; toujours créer les `GRANT` et politiques RLS dans la même migration qu'un `CREATE TABLE`.
4. **IA** : ajouter toute nouvelle intégration IA dans une Edge Function, jamais côté client.
5. **Nettoyage** : la racine du dépôt contient des scripts ponctuels historiques (`fix*.mjs`, `patch*.cjs...`) qui ne font pas partie du runtime et peuvent être archivés.

Conclusion

AcadémiePlus est une plateforme adaptative complète couvrant l'ensemble du parcours d'un élève : positionnement initial, cours, exercices/quizz calibrés, assistant IA, suivi de progression et reporting parental. Son architecture (React + Supabase + Edge Functions IA) sépare clairement présentation, logique métier pure et accès données, ce qui en facilite l'évolution.